

Does Financial Language Understanding Transfer to Market Prediction? A Systematic Study Across Models, Retrieval Paradigms, and Prediction Targets

Surya Suresh

Dheeraj Gosula

Nikhil Kumar

Abstract

Earnings call transcripts and SEC filings are rich sources of financial information, yet it remains unclear whether models that better understand this language also produce more reliable market predictions. We study this question through three coordinated components. First, we compare fine-tuning strategies (single-task, multi-task, and curriculum) across three model architectures on three financial language understanding tasks. Second, we evaluate retrieval-augmented generation (RAG) as an alternative to fine-tuning, comparing sparse, dense, and hybrid retrieval and testing whether retrieval can replace or complement domain adaptation. Third, we measure predictive transfer: whether gains in language understanding carry over to stock direction, earnings surprise, and volatility prediction, and whether this holds across market periods and industry sectors. Together, these components reveal when strong financial language understanding actually leads to better predictive performance.

1 Introduction

Earnings call transcripts and SEC annual reports convey financial information in natural language, and market participants rely on this language to form price expectations. However, prior NLP work has largely treated financial language understanding and market prediction as separate problems. It remains unclear whether a model that better extracts and reasons over financial text actually produces better predictions downstream. Our central research question is: *does improved financial language understanding reliably transfer to improved market signal quality, and if so, under what conditions?*

We break this into three sub-questions, each addressed by one research component. First, which fine-tuning strategies and architectures yield the strongest financial language understanding, and

does multi-task training help or hurt? Second, can retrieval-augmented generation serve as a viable alternative to fine-tuning for financial QA, and how does retrieval quality affect end-to-end performance? Third, do language understanding gains transfer uniformly across prediction targets, market conditions, and document types, or are they narrow and task-specific?

2 Related Work

Lexicon-based sentiment analysis (Loughran and McDonald, 2011) and domain-pretrained models like FinBERT (Araci, 2019) are the dominant approaches in financial NLP. Prior work has shown that earnings call language can predict market behavior (Qin and Yang, 2019), but transcripts are typically treated as flat documents without structural decomposition. Numerical reasoning benchmarks such as FinQA (Chen et al., 2021) and ConvFinQA (Chen et al., 2022) test financial comprehension but do not link understanding to prediction. Large-scale studies like BloombergGPT (Wu et al., 2023) explore financial pretraining but do not systematically compare architectures or fine-tuning strategies. RAG has been studied in open-domain settings (Lewis et al., 2020), but its effectiveness relative to fine-tuning in the financial domain, and the impact of retrieval strategy choice, remains unexplored. Multi-task learning has been studied broadly (Raffel et al., 2020), but not for the specific question of whether financial task co-training improves downstream predictive transfer.

3 Tasks, Data, and Evaluation

3.1 Language Understanding Tasks

We evaluate three tasks that capture different aspects of financial language comprehension.

Task 1: Numerical extraction. The model identifies structured financial values (revenue, EPS, guidance figures) and their attributes from tran-

script text. We evaluate with precision, recall, and F1 against manually verified annotations, distinguishing exact matches from partial credit to capture near-miss errors.

Task 2: Evidence-grounded QA. The model answers factual financial questions and cites the specific transcript spans supporting its answer. We measure answer accuracy, citation exact match, citation partial match, and hallucinated number rate. The hallucination metric checks whether numerical values in generated answers actually appear in the source document.

Task 3: Financial sentiment classification. The model assigns positive, negative, or neutral sentiment to financial statements. This is a standard benchmark that enables comparison with established baselines. We evaluate with macro-averaged F1.

3.2 Downstream Prediction Tasks

We test predictive transfer on three targets derived from market data. **Stock direction** predicts whether a company’s stock closes up or down in the trading window after an earnings call. **Earnings surprise direction** predicts whether reported EPS beats, meets, or misses analyst consensus, testing whether transcript signals capture firm-specific news beyond general market sentiment. **Post-call volatility** predicts whether realized return volatility in the five days after a call falls in the top quartile, testing whether model uncertainty signals reflect genuine investor uncertainty rather than just directional bias.

All targets are treated as classification problems. For each, we train a logistic regression classifier on signals derived from model outputs (sentiment scores, extracted guidance values, uncertainty markers, and QA-derived features). By holding the classifier constant across all language model conditions, we ensure that any differences in prediction quality are attributable to the language model, not the prediction architecture.

3.3 Datasets

The **lamini/earnings-calls-qa** corpus (~860,000 transcript QA pairs) provides training data for Tasks 1 and 2. The **Bose345/sp500_earnings_transcripts** corpus provides transcripts for prediction experiments, covering tens of thousands of calls across S&P 500 firms from 2010 to 2024. **Financial PhraseBank** (Malo et al., 2014) provides 4,840 annotated financial

sentences for Task 3. **FinanceBench** (Islam et al., 2023) provides QA pairs over 10-K, 10-Q, and 8-K filings and serves as our cross-domain evaluation benchmark. **FinQA** (Chen et al., 2021) and **ConvFinQA** (Chen et al., 2022) provide additional numerical reasoning evaluation. Stock direction and volatility labels come from historical prices via yfinance; earnings surprise labels come from analyst EPS consensus estimates via yahooquery. All splits are chronological to prevent data leakage.

3.4 Baselines

We compare against four baselines of increasing sophistication: (1) majority class prediction; (2) Loughran-McDonald financial sentiment dictionary (Loughran and McDonald, 2011) applied to full transcript text; (3) zero-shot FinBERT; and (4) a base instruction-tuned LLM with no fine-tuning or retrieval. All baselines are evaluated identically to the fine-tuned conditions across all three prediction targets.

4 Component A: Fine-Tuning Strategies and Architecture Comparison

4.1 Model Families

We evaluate three architecture families to test whether fine-tuning benefits are architecture-specific or general.

Encoder-only: ProsusAI/finbert (Araci, 2019), a BERT-based model pretrained on financial text. We fine-tune it with task-specific classification and span extraction heads. This represents strong domain pretraining with a discriminative architecture.

Encoder-decoder: google/flan-t5-base (Chung et al., 2022), fine-tuned with task-prefixed inputs. This lets us directly evaluate a generative seq2seq model on extraction and QA without the mismatch that comes from prompting alone.

Decoder-only: A publicly available instruction-tuned LLM (7B parameters), fine-tuned with LoRA (Hu et al., 2022) at rank $r \in \{4, 16, 64\}$. We also evaluate the base model with prompting only to isolate the effect of fine-tuning.

4.2 Fine-Tuning Regimes

For each architecture, we compare three training regimes:

Single-task fine-tuning. Each model is trained separately on each task, providing per-task base-

lines and an upper bound for task-specific optimization.

Joint multi-task fine-tuning. All three tasks are trained simultaneously with uniform or temperature-scaled sampling. This tests whether the tasks benefit from shared representations.

Curriculum fine-tuning. Tasks are introduced in a fixed order, building from simpler (sentiment) to more complex (QA). We test two orderings: sentiment $\rightarrow$ extraction $\rightarrow$ QA and extraction $\rightarrow$ sentiment $\rightarrow$ QA.

4.3 Ablation Studies

For the decoder-only model, we ablate LoRA rank ($r \in \{4, 16, 64\}$), training data size (10%, 25%, 50%, 100%), and task ordering in curriculum fine-tuning. For the encoder-only model, we ablate layer freezing (bottom $k \in \{0, 6, 9\}$ transformer blocks). These ablations document how sensitive financial fine-tuning is to common engineering choices.

4.4 Cross-Document-Domain Transfer

All fine-tuned models are evaluated zero-shot on FinanceBench, which contains QA over SEC filings not seen during training. This tests whether understanding learned from earnings transcripts generalizes to regulatory filings. We also fine-tune on a 50/50 mixed corpus of transcripts and SEC filings and compare cross-domain performance before and after mixed training.

5 Component B: Retrieval-Augmented Generation for Financial QA

5.1 Motivation

Fine-tuning bakes domain knowledge into model weights, which become static after training. RAG instead stores knowledge in an external index that can be updated without retraining. For financial applications, where facts change every quarter and documents are long, RAG may be more practical. This component tests whether retrieval can replace or complement fine-tuning on the QA task.

5.2 Retrieval Strategies

We compare three retrieval approaches:

Sparse retrieval (BM25). Classical bag-of-words retrieval over transcript passages. This is a strong lexical baseline, especially for numerical entities that have little paraphrase variation.

Dense retrieval. Passages are embedded with sentence-transformers/all-MiniLM-L6-v2 (Reimers and Gurevych, 2019) and indexed with FAISS. At inference, queries are embedded and the top- k passages are retrieved by cosine similarity.

Hybrid retrieval. BM25 and dense scores are combined via linear interpolation with weight α , tuned over $\alpha \in \{0.2, 0.4, 0.6, 0.8\}$ on a development set.

For all strategies, we measure top- k retrieval recall at $k \in \{1, 3, 5, 10\}$ against gold supporting spans. Retrieved passages are prepended as context for the reader model.

5.3 Reader Models and RAG Conditions

We evaluate four conditions on the QA task:

1. Base LLM, no retrieval, no fine-tuning (zero-shot prompting only)
2. Base LLM with the best retrieval strategy (RAG only)
3. Fine-tuned LLM, no retrieval (fine-tuning only)
4. Fine-tuned LLM with the best retrieval strategy (RAG + fine-tuning)

This 2×2 design isolates the contributions of retrieval and fine-tuning and reveals whether they are additive, redundant, or complementary.

5.4 Retrieval Failure Analysis

When the system produces an incorrect answer, we classify the failure as either (1) *retrieval failure*, where the correct passage was not retrieved, or (2) *reader failure*, where the correct passage was retrieved but the model still answered incorrectly. This tells us whether improvement efforts should focus on retrieval or comprehension. We report the breakdown by question type (numerical, factual, guidance-related) and document section.

5.5 Long-Document Retrieval

Earnings transcripts can exceed 10,000 tokens. We study how passage chunking granularity (sentence-level, 128-token, 256-token, 512-token windows) affects retrieval recall and end-to-end QA accuracy, providing practical guidance for applying RAG to long financial documents.

6 Component C: Predictive Transfer and Robustness Evaluation

6.1 Feature Extraction and Signal Construction

From each model condition in Components A and B, we extract a fixed set of signals from transcripts: (1) overall and section-level sentiment scores (prepared remarks and analyst Q&A scored separately); (2) numerical guidance signals (extracted revenue/EPS guidance values and year-over-year direction); (3) uncertainty markers (hedging language frequency and modal verb counts from QA and extraction outputs); and (4) QA-derived confidence signals (answer length, citation coverage, and hallucination rate as a proxy for model uncertainty). These features are constructed identically across all conditions so that prediction differences reflect language understanding quality, not feature engineering.

6.2 Prediction Models and Evaluation

For each prediction target (stock direction, earnings surprise, volatility), we train logistic regression classifiers on the extracted signals and evaluate with AUC-ROC, accuracy, balanced accuracy, and expected calibration error. We compute confidence intervals via bootstrap resampling ($n = 1,000$) and test for significant AUC improvements using paired permutation tests.

To put our results in context, we compare against prior reported AUC values for earnings call prediction (Qin and Yang, 2019). We also compare predictions from Q&A sections alone versus prepared remarks alone, testing whether spontaneous analyst dialogue carries more predictive signal than scripted management statements.

6.3 Temporal Robustness

We partition the test data into three market-period buckets: pre-COVID (2015–2019), COVID (2020–2021), and post-COVID (2022–2024), training only on data preceding each evaluation period. We evaluate all model conditions on each bucket and plot performance over time to identify degradation patterns. We also analyze vocabulary shift, identifying financial terminology that emerges in later periods (e.g., pandemic-era hedging language, supply chain terms), and test whether models trained on earlier data handle novel vocabulary.

6.4 Sector-Stratified and Firm-Size Stratified Analysis

We stratify prediction results by GICS sector (technology, healthcare, financials, industrials, and others) and by market capitalization (large cap vs. mid cap). This tests whether language understanding gains are concentrated in certain firm types; for instance, whether fine-tuning helps more for sectors with specialized language (healthcare regulatory terminology, technology forward guidance) than for sectors with more standardized reporting.

6.5 Structured Error Analysis

We categorize errors across all tasks and prediction targets. For extraction: wrong number, wrong entity, hallucinated value, or missed mention. For QA: wrong span, unsupported citation, hallucination, or correct answer with wrong citation. For prediction: errors are broken down by target, sector, market cap tier, and model condition. Comparing error profiles across architectures and training regimes reveals whether different design choices lead to systematically different failure modes.

7 Work Plan

Component A: Architecture and Fine-Tuning.

Covers the full data pipeline (transcript preprocessing, section segmentation, SEC filing extraction from FinanceBench, training example construction, and chronological splits), fine-tuning of all three model families across all tasks and training regimes, LoRA rank and layer-freezing ablations, and cross-document-domain transfer experiments.

Component B: Retrieval-Augmented Generation.

Covers the RAG pipeline (passage chunking, BM25 indexing via rank_bm25, dense indexing via FAISS and sentence-transformers, and hybrid retrieval with interpolation), evaluation of all retrieval conditions on the QA task, retrieval failure analysis, and the chunking granularity ablation.

Component C: Prediction and Robustness.

Covers the signal extraction pipeline and logistic regression classifiers for all three prediction targets, bootstrap confidence intervals and permutation tests, temporal robustness evaluation, sector- and firm-size-stratified analysis, and the structured error analysis pipeline.

Integration and shared milestones. All team members contribute to a unified evaluation frame-

work connecting Components A and B to Component C. The project proceeds in three overlapping phases: first, data pipeline setup, shared infrastructure, and baseline implementations; second, primary experiments for all three components in parallel; and third, targeted ablations, cross-component analysis, error analysis, and writing.

8 Progress Report

8.1 Infrastructure and Data Pipeline

Since the proposal, we have built and validated the full end-to-end pipeline for Component C. The code is organized as a Python package under `src/`, with separate modules for data loading, text preprocessing, feature extraction, classification, and evaluation. All experiments run on the Ohio Supercomputer Center (OSC) Pitzer cluster via a GPU-enabled Slurm job script. The environment is fully reproducible with pinned dependencies: PyTorch 2.3.1 (CUDA 11.8 for V100/sm_70 compatibility), Transformers 4.44.2, and supporting libraries.

The data pipeline loads transcripts from the **Bose345/sp500_earnings_transcripts** corpus and fetches stock price labels via `yfinance`, computing a binary direction label from closing price movement over the three trading days after each call. From the 500 most recent records, 493 yielded valid labels after excluding delisted or renamed tickers (PARA, BF.B, MMC, DFS, FI, IPG). Using a strict chronological split with no shuffling, we obtain 394 training and 99 test examples. The labels are moderately imbalanced: 309 positive (price up) vs. 184 negative (price down), reflecting the general upward drift of S&P 500 constituents over the sampling window.

The preprocessing module uses a regex-based section splitter that detects Q&A boundaries via patterns like “*question-and-answer session*” and “*open the floor to questions*”, dividing each transcript into a prepared remarks segment and a Q&A segment. This segmentation is central to the improved feature condition described below.

8.2 Preliminary Baseline Results

We have run the first two experimental conditions from Component C: a **baseline** using full-transcript FinBERT sentiment and an **improved** condition using section-separated sentiment scores. Both use a logistic regression classifier with balanced class weights, and all hyperparameters are held constant across conditions.

Model	Accuracy	AUC-ROC
Majority class	0.6263	0.5000
Baseline (full transcript)	0.5152	0.4883
Improved (prepared + Q&A)	0.5152	0.5122

Table 1: Preliminary results on 99 held-out test examples. Majority class accuracy reflects the label imbalance (62.6% positive). AUC-ROC is the primary metric; accuracy is reported with balanced class weights.

Baseline condition. A single sentiment score is computed over the full transcript using ProsusAI/finbert (Araci, 2019). Transcripts are chunked into 150-word windows to fit within the 512-token limit, and the score is defined as $P(\text{positive}) - P(\text{negative})$.

Improved condition. Three sentiment scores are computed separately from (1) the full transcript, (2) the prepared remarks segment, and (3) the analyst Q&A segment. This three-dimensional feature vector tests whether breaking the transcript into sections provides information that the aggregate score misses.

Results on the 99-example held-out set are shown in Table 1.

8.3 Analysis of Preliminary Results

Three observations stand out from these initial results.

First, the baseline AUC of 0.4883 is essentially at chance, confirming that aggregate sentiment over a full transcript provides no useful directional signal. This aligns with prior findings that management language in prepared remarks tends to be consistently positive regardless of actual performance (Qin and Yang, 2019), suggesting that treating the transcript as a single undifferentiated block conflates qualitatively different sources of language.

Second, the improved condition yields AUC 0.5122, a modest +0.0239 gain over the baseline. While small in absolute terms, this is directionally consistent with the hypothesis that section-level disaggregation recovers signal masked by the aggregate score. We also observed a notable timing asymmetry: prepared remarks sentiment was computed in under one minute for 394 examples, while full-transcript and Q&A inference each took roughly 22 minutes on a V100 GPU. This reflects the fact that many transcripts have short or empty prepared remarks segments under the current regex splitter, an issue we plan to address

next.

Third, the label imbalance ($\approx 62\%/38\%$) is significant. The majority class baseline achieves 62.6% accuracy simply by predicting all positives, outperforming both model conditions. This reflects the upward bias inherent in sampling recent S&P 500 calls. We will address this by expanding the sampling window to include earlier, more varied market periods, consistent with the temporal robustness analysis planned in Component C.

8.4 Revised Plan for Remaining Work

These preliminary experiments have validated the core infrastructure and pointed to four concrete next steps.

Improving section segmentation. The current regex splitter works well when transcripts use standard transition phrases, but roughly 40% of transcripts yield an empty prepared remarks segment, adding no differential signal. We will add more detection patterns and fall back to a heuristic positional split (first 60% as prepared remarks) when no explicit marker is found.

Expanding the feature set. So far we use only sentiment scores. Next, we will add the guidance and uncertainty features described in the proposal: extracted EPS and revenue guidance direction (year-over-year sign), modal verb density as a hedging proxy, and forward-looking statement frequency. These can all be computed from the existing pipeline without fine-tuned models.

Connecting Components A and B to C. The current results establish a lower bound using zero-shot FinBERT. The key question is whether fine-tuning (Component A) or retrieval augmentation (Component B) improves these signals. Our first step will be single-task fine-tuning of FinBERT on Financial PhraseBank, replacing the zero-shot scores with fine-tuned probabilities and measuring the resulting AUC change. This directly tests whether better language understanding leads to better predictions, which is the core question of the project.

Temporal stratification. The current 80/20 chronological split is not yet broken down by market period. We will implement pre-COVID, COVID, and post-COVID evaluation buckets as described in Section 6.3, using the full 33,362-transcript corpus instead of the 500-record sample. The larger corpus also helps reduce label imbalance by including periods with more varied return distributions.

References

- Dogu Araci. 2019. FinBERT: Financial Sentiment Analysis with Pre-trained Language Models. *arXiv preprint arXiv:1908.10063*.
- Zhuang Chen et al. 2021. FinQA: A Dataset of Numerical Reasoning over Financial Data. In *EMNLP*, pages 3697–3711.
- Zhuang Chen et al. 2022. ConvFinQA: Exploring the Chain of Numerical Reasoning in Conversational Finance Question Answering. In *EMNLP*, pages 6279–6292.
- Hyung Won Chung et al. 2022. Scaling Instruction-Finetuned Language Models. *arXiv preprint arXiv:2210.11416*.
- Edward Hu et al. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR*.
- Zain Ul Islam Fazal et al. 2023. FinanceBench: A New Benchmark for Financial Question Answering. *arXiv preprint arXiv:2311.11944*.
- Patrick Lewis et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*.
- Tim Loughran and Bill McDonald. 2011. When Is a Liability Not a Liability? *Journal of Finance*, 66(1):35–65.
- Pekka Malo et al. 2014. Good Debt or Bad Debt: Detecting Semantic Orientations in Economic Texts. *Journal of the American Society for Information Science and Technology*, 65(4):782–796.
- Yao Qin and Yi Yang. 2019. What You Say and How You Say It Matters: Predicting Stock Volatility Using Verbal and Vocal Cues. In *ACL*, pages 390–401.
- Colin Raffel et al. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21(140):1–67.
- Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *EMNLP*, pages 3982–3992.
- Shijie Wu et al. 2023. BloombergGPT: A Large Language Model for Finance. *arXiv preprint arXiv:2303.17564*.